Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №3 по курсу

«Операционные системы»

Группа: М8О-211БВ-25

Студент: Самсонов И.А.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 27.10.26

Москва, 2026

Постановка задачи

Вариант 18. Цель работы — освоить обмен данными между процессами через именованную разделяемую память и её отображение в виртуальное адресное пространство, а также синхронизацию чтения и записи семафорами.

Родительский процесс создаёт два дочерних процесса. Первые две строки ввода задают имена выходных файлов. Нечётные строки текста передаются первому потомку, чётные — второму. Дочерние процессы удаляют латинские гласные A, E, I, O, U в обоих регистрах и записывают результат в назначенные файлы. Буквы Y и y, русские буквы и остальные байты сохраняются, как в первой лабораторной.

Согласно уточнениям задания, объект IPC создаётся через shm_open; дескриптор именно этого объекта передаётся в mmap. Для синхронизации используются именованные POSIX-семафоры. Имена ресурсов формируются отдельно для каждой пары процессов. Обычные файлы в /tmp и устаревший System V API для обмена данными не применяются.

Общий метод и алгоритм решения

Решение состоит из программ parent и child и общего модуля ipc. Родитель открывает два выходных файла, проверяет, что это разные обычные файлы, создаёт два канала на основе shared memory и запускает двух потомков через fork и execl. Стандартный вывод каждого потомка перенаправляется в его файл через dup2; имена IPC-объектов передаются программе child в аргументах командной строки.

Для каждого потомка создаётся отдельный общий слот struct shared_block. Он содержит length, признак конца ввода end_of_input и массив data размером 4096 байт. Обычных указателей в передаваемой структуре нет: адреса отображений у процессов могут различаться. Служебная struct channel с локальными указателями и флагами владения хранится отдельно в памяти каждого процесса.

Shared memory задаёт общий объект данных, а memory mapping предоставляет процессу доступ к нему через виртуальные адреса. Родитель выполняет shm_open с O_CREAT | O_EXCL | O_RDWR и правами 0600, затем ftruncate и mmap с PROT_READ | PROT_WRITE и MAP_SHARED. После успешного mmap дескриптор закрывается: отображение остаётся действительным. После exec потомок заново открывает объект по имени и создаёт собственное отображение.

Имена имеют вид /l3_<PID hex>_<наносекунды hex>_<номер><тип>, где m обозначает shared memory, e — empty, r — ready. PID различает одновременно работающих родителей, номер различает двух потомков. O_EXCL не допускает подключения к уже существующему чужому объекту при совпадении имени. Всего используются два объекта общей памяти и четыре семафора.

Синхронизация чтения и записи

Для каждого слота семафор empty создаётся со значением 1, ready — со значением 0. Родитель получает empty, заполняет data, length и end_of_input, затем вызывает sem_post(ready). Потомок ждёт sem_wait(ready), читает сообщение, удаляет гласные и записывает результат. Только после обработки он вызывает sem_post(empty), разрешая следующую запись.

```
parent: получить empty -> заполнить слот -> sem_post(ready)
child:  sem_wait(ready) -> обработать и записать -> sem_post(empty)
```

Получение empty в родителе реализовано через sem_trywait, проверку waitpid с WNOHANG и nanosleep на 1 мс при EAGAIN. Это позволяет обнаружить завершение потребителя, который уже не сможет вернуть семафор. Потомок блокируется в sem_wait. Семафоры задают порядок доступа к общему слоту; volatile и msync не заменяют этот протокол.

Родитель переключает получателя только после символа LF. При заполнении буфера строка отправляется частями одному и тому же потомку, поэтому её длина не ограничена 4096 байтами. После EOF сначала передаётся остаток текста, затем каждому потомку отправляется отдельное сообщение length = 0, end_of_input = 1. Закрытие дескриптора shared memory само по себе не является сигналом конца ввода.

Использованные функции и обработка ошибок

shm_open и shm_unlink создают объект разделяемой памяти и удаляют его имя; ftruncate задаёт размер; mmap и munmap создают и снимают отображение. sem_open, sem_wait, sem_trywait, sem_post, sem_close и sem_unlink управляют именованными семафорами. fork и execl создают и запускают потомков; open, fstat, dup2, read, write и close обслуживают ввод и выходные файлы; waitpid проверяет завершение; sigaction задаёт обработку SIGINT и SIGTERM.

Проверяются ошибки создания ресурсов, отображения, запуска процессов и вывода. При EINTR допустимые операции повторяются; write_all учитывает частичную запись. Перед обнулением выходных файлов сравниваются st_dev и st_ino, поэтому два имени одного файла отклоняются. O_NONBLOCK предотвращает блокировку на случайно переданном FIFO. Потомок проверяет длину сообщения и формат EOF.

Родитель удаляет только созданные им IPC-имена; каждый процесс закрывает свои отображения и семафоры. При ошибке оставшиеся собственные потомки завершаются и собираются waitpid. Обработчики SIGINT/SIGTERM только устанавливают флаг, очистка выполняется обычным кодом. SIGKILL родителя перехватить нельзя: при таком завершении могут остаться именованные объекты и ожидающие потомки.

Код программы

ipc.h

```
#ifndef LAB3_IPC_H
#define LAB3_IPC_H

#include <semaphore.h>
#include <stddef.h>

#define BUFFER_SIZE 4096
#define IPC_NAME_SIZE 32

/* No pointers: mappings in different processes can have different addresses. */
struct shared_block {
    size_t length;
    int end_of_input;
    unsigned char data[BUFFER_SIZE];
};

struct channel {
    char memory_name[IPC_NAME_SIZE];
    char empty_name[IPC_NAME_SIZE];
    char ready_name[IPC_NAME_SIZE];
    struct shared_block *block;
    sem_t *empty;
    sem_t *ready;
    int owns_memory;
    int owns_empty;
    int owns_ready;
};

void channel_init(struct channel *channel);
int channel_create(struct channel *channel, unsigned int number);
int channel_open(struct channel *channel, const char *memory_name,
                 const char *empty_name, const char *ready_name);
int channel_close(struct channel *channel);
int write_all(int fd, const void *buffer, size_t length);
```

```
#endif
```

ipc.c

```
#define _POSIX_C_SOURCE 200809L
#include "ipc.h"

#include <errno.h>
#include <fcntl.h>
#include <stdio.h>
#include <string.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <time.h>
#include <unistd.h>

void channel_init(struct channel *channel) {
    memset(channel, 0, sizeof(*channel));
    channel->block = MAP_FAILED;
    channel->empty = SEM_FAILED;
    channel->ready = SEM_FAILED;
}

static int map_memory(struct channel *channel, int fd) {
    channel->block = mmap(NULL, sizeof(struct shared_block),
                          PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
    int saved_errno = errno;
    int close_result = close(fd);
    if (channel->block == MAP_FAILED) {
        errno = saved_errno;
        return -1;
    }
    return close_result;
}

static int make_name(char *name, unsigned int number, char kind,
                     unsigned long stamp) {
    int length = snprintf(name, IPC_NAME_SIZE, "/l3_%lx_%lx_%u%c",
                          (unsigned long)getpid(), stamp, number, kind);
    if (length < 0 || length >= IPC_NAME_SIZE) {
        errno = ENAMETOOLONG;
        return -1;
    }
    return 0;
}

int channel_create(struct channel *channel, unsigned int number) {
    struct timespec now;
    if (clock_gettime(CLOCK_MONOTONIC, &now) == -1) return -1;
    unsigned long stamp = (unsigned long)now.tv_nsec;
    if (make_name(channel->memory_name, number, 'm', stamp) == -1 ||
        make_name(channel->empty_name, number, 'e', stamp) == -1 ||
        make_name(channel->ready_name, number, 'r', stamp) == -1) return -1;

    int fd = shm_open(channel->memory_name, O_CREAT | O_EXCL | O_RDWR, 0600);
    if (fd == -1) return -1;
    channel->owns_memory = 1;
    if (ftruncate(fd, (off_t)sizeof(struct shared_block)) == -1) {
        int saved_errno = errno;
        close(fd);
        errno = saved_errno;
        return -1;
    }
    if (map_memory(channel, fd) == -1) return -1;
    channel->empty = sem_open(channel->empty_name, O_CREAT | O_EXCL, 0600, 1);
    if (channel->empty == SEM_FAILED) return -1;
    channel->owns_empty = 1;
    channel->ready = sem_open(channel->ready_name, O_CREAT | O_EXCL, 0600, 0);
    if (channel->ready == SEM_FAILED) return -1;
    channel->owns_ready = 1;
    return 0;
}

int channel_open(struct channel *channel, const char *memory_name,
                 const char *empty_name, const char *ready_name) {
    int fd = shm_open(memory_name, O_RDWR, 0);
```

```
    if (fd == -1) return -1;
    struct stat info;
    int result = fstat(fd, &info);
    /* Darwin rounds a shared object's size up to a whole memory page. */
    if (result == -1 || info.st_size < (off_t)sizeof(struct shared_block)) {
        int saved_errno = result == -1 ? errno : EINVAL;
        close(fd);
        errno = saved_errno;
        return -1;
    }
    if (map_memory(channel, fd) == -1) return -1;
    channel->empty = sem_open(empty_name, 0);
    if (channel->empty == SEM_FAILED) return -1;
    channel->ready = sem_open(ready_name, 0);
    return channel->ready == SEM_FAILED ? -1 : 0;
}

int channel_close(struct channel *channel) {
    int result = 0;
    if (channel->block != MAP_FAILED &&
        munmap(channel->block, sizeof(struct shared_block)) == -1) {
        perror("munmap");
        result = -1;
    }
    if (channel->empty != SEM_FAILED && sem_close(channel->empty) == -1) {
        perror("sem_close(empty)");
        result = -1;
    }
    if (channel->ready != SEM_FAILED && sem_close(channel->ready) == -1) {
        perror("sem_close(ready)");
        result = -1;
    }
    if (channel->owns_memory && shm_unlink(channel->memory_name) == -1) {
        perror("shm_unlink");
        result = -1;
    }
    if (channel->owns_empty && sem_unlink(channel->empty_name) == -1) {
        perror("sem_unlink(empty)");
        result = -1;
    }
    if (channel->owns_ready && sem_unlink(channel->ready_name) == -1) {
        perror("sem_unlink(ready)");
        result = -1;
    }
    channel_init(channel);
    return result;
}

int write_all(int fd, const void *buffer, size_t length) {
    const unsigned char *bytes = buffer;
    while (length > 0) {
        ssize_t written = write(fd, bytes, length);
        if (written == -1 && errno == EINTR) continue;
        if (written <= 0) {
            if (written == 0) errno = EIO;
            return -1;
        }
        bytes += written;
        length -= (size_t)written;
    }
    return 0;
}
```

parent.c

```
#define _POSIX_C_SOURCE 200809L
#include "ipc.h"

#include <errno.h>
#include <fcntl.h>
#include <poll.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/stat.h>
```

```
#include <sys/wait.h>
#include <time.h>
#include <unistd.h>

#define FILE_NAME_SIZE 4096

static volatile sig_atomic_t stop_requested = 0;

static void request_stop(int signal_number) {
    stop_requested = signal_number;
}

static ssize_t read_input(void *buffer, size_t length) {
    for (;;) {
        if (stop_requested) {
            errno = EINTR;
            return -1;
        }
        /* A timeout bounds the signal-before-wait race even on an idle stdin. */
        struct pollfd input = {STDIN_FILENO, POLLIN, 0};
        int ready = poll(&input, 1, 100);
        if (ready == -1 && errno == EINTR) continue;
        if (ready == -1) return -1;
        if (ready == 0) continue;
        if (stop_requested) continue;
        ssize_t count = read(STDIN_FILENO, buffer, length);
        if (count == -1 && errno == EINTR) continue;
        return count;
    }
}

static int read_file_name(char *name) {
    size_t length = 0;
    unsigned char character;
    for (;;) {
        ssize_t count = read_input(&character, 1);
        if (count == -1) return -1;
        if (count == 0 || character == '\n') break;
        if (character == '\0') {
            errno = EINVAL;
            return -1;
        }
        if (length == FILE_NAME_SIZE - 1) {
            errno = ENAMETOOLONG;
            return -1;
        }
        name[length++] = (char)character;
    }
    name[length] = '\0';
    if (length == 0) {
        errno = EINVAL;
        return -1;
    }
    return 0;
}

/* A bounded sleep also lets us detect a consumer that died without posting.
 * sem_timedwait is not available on macOS; the child uses blocking sem_wait. */
static int wait_empty(struct channel *channel, pid_t *child) {
    for (;;) {
        if (stop_requested) {
            errno = EINTR;
            return -1;
        }
        int status;
        pid_t ended = waitpid(*child, &status, WNOHANG);
        if (ended == *child) {
            *child = -1;
            errno = EPIPE;
            return -1;
        }
        if (ended == -1) {
            if (errno == EINTR) continue;
            return -1;
        }
```

```
        if (sem_trywait(channel->empty) == 0) return 0;
        if (errno == EINTR) continue;
        if (errno != EAGAIN) return -1;
        struct timespec pause = {0, 1000000L};
        if (nanosleep(&pause, NULL) == -1 && errno != EINTR) return -1;
    }
}

static int send_block(struct channel *channel, pid_t *child,
                      const unsigned char *data, size_t length, int end) {
    if (wait_empty(channel, child) == -1) return -1;
    if (length != 0) memcpy(channel->block->data, data, length);
    channel->block->length = length;
    channel->block->end_of_input = end;
    return sem_post(channel->ready);
}

static pid_t start_child(struct channel *channel, const int output[2], int index) {
    pid_t child = fork();
    if (child != 0) return child;
    struct sigaction action = {0};
    action.sa_handler = SIG_DFL;
    sigemptyset(&action.sa_mask);
    if (sigaction(SIGINT, &action, NULL) == -1 ||
        sigaction(SIGTERM, &action, NULL) == -1 || stop_requested) _exit(1);
    if (dup2(output[index], STDOUT_FILENO) == -1) {
        perror("child: dup2");
        _exit(1);
    }
    for (int i = 0; i < 2; ++i) {
        if (output[i] != STDOUT_FILENO) close(output[i]);
    }
    execl("./child", "child", channel->memory_name, channel->empty_name,
          channel->ready_name, (char *)NULL);
    perror("child: execl ./child");
    _exit(1);
}

int main(void) {
    struct channel channels[2];
    pid_t children[2] = {-1, -1};
    int output[2] = {-1, -1};
    int result = EXIT_FAILURE;
    for (int i = 0; i < 2; ++i) channel_init(&channels[i]);
    struct sigaction action = {0};
    action.sa_handler = request_stop;
    sigemptyset(&action.sa_mask);
    if (sigaction(SIGINT, &action, NULL) == -1 ||
        sigaction(SIGTERM, &action, NULL) == -1) {
        perror("sigaction");
        goto cleanup;
    }
    char names[2][FILE_NAME_SIZE];
    const char *prompts[2] = {"First file name: ", "Second file name: "};
    for (int i = 0; i < 2; ++i) {
        if (write_all(STDERR_FILENO, prompts[i], strlen(prompts[i])) == -1 ||
            read_file_name(names[i]) == -1) {
            perror("read file name");
            goto cleanup;
        }
    }
    struct stat info[2];
    for (int i = 0; i < 2; ++i) {
        output[i] = open(names[i], O_WRONLY | O_CREAT | O_NONBLOCK, 0644);
        if (output[i] == -1 || fstat(output[i], &info[i]) == -1) {
            perror("open output");
            goto cleanup;
        }
        if (!S_ISREG(info[i].st_mode)) {
            fprintf(stderr, "Output must be a regular file\n");
            goto cleanup;
        }
    }
    if (info[0].st_dev == info[1].st_dev && info[0].st_ino == info[1].st_ino) {
        fprintf(stderr, "Output files must be different (including hard links)\n");
```

```
    goto cleanup;
}
for (int i = 0; i < 2; ++i) {
    if (ftruncate(output[i], 0) == -1 ||
        channel_create(&channels[i], (unsigned int)i) == -1) {
        perror("prepare output / IPC");
        goto cleanup;
    }
}
for (int i = 0; i < 2; ++i) {
    children[i] = start_child(&channels[i], output, i);
    if (children[i] == -1) {
        perror("fork");
        goto cleanup;
    }
}
for (int i = 0; i < 2; ++i) {
    int fd = output[i];
    output[i] = -1;
    if (close(fd) == -1) {
        perror("close output in parent");
        goto cleanup;
    }
}
unsigned char input[BUFFER_SIZE], fragment[BUFFER_SIZE];
size_t used = 0;
int destination = 0;
for (;;) {
    ssize_t count = read_input(input, sizeof(input));
    if (count == -1) {
        perror("read input");
        goto cleanup;
    }
    if (count == 0) break;
    for (ssize_t i = 0; i < count; ++i) {
        fragment[used++] = input[i];
        if (used == BUFFER_SIZE || input[i] == '\n') {
            if (send_block(&channels[destination], &children[destination],
                           fragment, used, 0) == -1) {
                perror("send data (consumer may have exited)");
                goto cleanup;
            }
            used = 0;
        }
        if (input[i] == '\n') destination = 1 - destination;
    }
}
if (used != 0 && send_block(&channels[destination], &children[destination],
                            fragment, used, 0) == -1) {
    perror("send final fragment");
    goto cleanup;
}
for (int i = 0; i < 2; ++i) {
    if (send_block(&channels[i], &children[i], NULL, 0, 1) == -1) {
        perror("send EOF");
        goto cleanup;
    }
}
result = EXIT_SUCCESS;
for (int i = 0; i < 2; ++i) {
    int status;
    pid_t ended;
    for (;;) {
        if (stop_requested) {
            errno = EINTR;
            ended = -1;
            break;
        }
        ended = waitpid(children[i], &status, WNOHANG);
        if (ended > 0 || (ended == -1 && errno != EINTR)) break;
        struct timespec pause = {0, 1000000L};
        if (nanosleep(&pause, NULL) == -1 && errno != EINTR) {
            ended = -1;
            break;
        }
```

```
        }
        if (ended == -1) {
            perror("waitpid");
            result = EXIT_FAILURE;
            goto cleanup;
        }
        children[i] = -1;
        if (!WIFEXITED(status) || WEXITSTATUS(status) != 0) {
            fprintf(stderr, "Child %d failed\n", i + 1);
            result = EXIT_FAILURE;
        }
    }
cleanup:
    for (int i = 0; i < 2; ++i) {
        if (children[i] > 0) {
            /* A stopped or broken consumer must not make error cleanup hang. */
            if (kill(children[i], SIGKILL) == -1 && errno != ESRCH) perror("kill");
            while (waitpid(children[i], NULL, 0) == -1 && errno == EINTR) {}
        }
        if (output[i] != -1 && close(output[i]) == -1) result = EXIT_FAILURE;
        if (channel_close(&channels[i]) == -1) result = EXIT_FAILURE;
    }
    if (stop_requested) result = EXIT_FAILURE;
    return result;
}
```

child.c

```
#define _POSIX_C_SOURCE 200809L
#include "ipc.h"

#include <errno.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

static int is_vowel(unsigned char character) {
    return character == 'a' || character == 'e' || character == 'i' ||
           character == 'o' || character == 'u' || character == 'A' ||
           character == 'E' || character == 'I' || character == 'O' ||
           character == 'U';
}

int main(int argc, char *argv[]) {
    if (argc != 4) {
        fprintf(stderr, "Usage: %s /memory /empty /ready\n", argv[0]);
        return EXIT_FAILURE;
    }
    struct channel channel;
    channel_init(&channel);
    int result = EXIT_FAILURE;
    if (channel_open(&channel, argv[1], argv[2], argv[3]) == -1) {
        perror("child: open IPC");
        goto cleanup;
    }
    for (;;) {
        int status;
        do {
            status = sem_wait(channel.ready);
        } while (status == -1 && errno == EINTR);
        if (status == -1) {
            perror("child: sem_wait");
            break;
        }
        size_t length = channel.block->length;
        int end_of_input = channel.block->end_of_input;
        if (length > BUFFER_SIZE || (end_of_input != 0 && end_of_input != 1) ||
            (end_of_input && length != 0)) {
            fprintf(stderr, "child: invalid shared-memory message\n");
            break;
        }
        unsigned char output[BUFFER_SIZE];
        size_t output_length = 0;
        for (size_t i = 0; i < length; ++i) {
            if (!is_vowel(channel.block->data[i])) {
                output[output_length++] = channel.block->data[i];
```

```
            }
        }
        if (write_all(STDOUT_FILENO, output, output_length) == -1) {
            perror("child: write output");
            break;
        }
        if (sem_post(channel.empty) == -1) {
            perror("child: sem_post");
            break;
        }
        if (end_of_input) {
            result = EXIT_SUCCESS;
            break;
        }
    }
cleanup:
    if (channel_close(&channel) == -1) result = EXIT_FAILURE;
    /* Report errors delayed until close (for example, a full filesystem). */
    if (close(STDOUT_FILENO) == -1) {
        perror("child: close output");
        result = EXIT_FAILURE;
    }
    return result;
}
```

Makefile

```
CC = cc
CPPFLAGS = -D_POSIX_C_SOURCE=200809L
CFLAGS = -std=c11 -Wall -Wextra -Wpedantic -Werror -O2
LDLIBS = -pthread

ifeq ($(shell uname -s),Linux)
LDLIBS += -lrt
endif

.PHONY: all test sanitize clean
all: parent child

parent: parent.c ipc.c ipc.h
	$(CC) $(CPPFLAGS) $(CFLAGS) $(LDFLAGS) parent.c ipc.c -o $@ $(LDLIBS)

child: child.c ipc.c ipc.h
	$(CC) $(CPPFLAGS) $(CFLAGS) $(LDFLAGS) child.c ipc.c -o $@ $(LDLIBS)

test: all
	python3 tests/test_lab3.py

sanitize:
	$(MAKE) clean
	$(MAKE) CFLAGS='-std=c11 -Wall -Wextra -Wpedantic -Werror -O1 -g -
fsanitize=address,undefined -fno-omit-frame-pointer' LDFLAGS='-fsanitize=address,undefined' test

clean:
	rm -f parent child
```

Протокол работы программы

Исходники из каталога OS_lab_1/OS_lab_3 собраны с C11 и строгими предупреждениями. Для основного теста использованы четыре строки из примера первой лабораторной. Нечётные строки должны попасть в odd.txt, чётные — в even.txt. Ниже приведены команда и фактически полученные результаты; вход передаётся перенаправлением из input.txt.

Сборка и входные данные

```
$ make
$ cat input.txt
odd.txt
even.txt
Operating Systems
Queue audio
Process communication
Fork and pipe
$ ./parent < input.txt
First file name: Second file name:
```

Приглашения печатаются в stderr. Результаты сохраняются только в выходных файлах; стандартный вывод родителя в этом запуске пуст. Код завершения parent равен 0.

Полученные результаты

```
$ cat odd.txt
prtng Systms
Prcss cmmnctn
$ cat even.txt
Q d
Frk nd pp
```

Выходные строки совпали с ожидаемым удалением латинских гласных; переводы строк сохранены. Дополнительно выполнены 23 интеграционные проверки текущих исходников в изолированной рабочей копии. Все проверки прошли на macOS arm64 с Apple Clang 21 и в Ubuntu 24.04 на aarch64 с GCC 13.3.0.

Проверки охватывают пустой ввод, пустые строки, EOF без завершающего LF, строки длиннее буфера, границы 4096 байт, UTF-8 и нулевые байты, случайные данные и перезапись файлов. Отдельно проверены 16 одновременных запусков, неверные пути и имена, совпадающие файлы и hard links, FIFO, отсутствие child, смерть потребителя, SIGINT/SIGTERM и удаление всех наблюдавшихся IPC-имён. В каталоге проверки /dev/shm после обычного Linux-запуска не осталось объектов.

```
Ran 23 tests
OK
```

Трассировка системных вызовов

Трассировка получена при реальном запуске тех же исходников в Ubuntu 24.04.5 LTS, Linux aarch64, внутри Docker. Сборка выполнена GCC 13.3.0 с флагами из Makefile; для Linux подключены -pthread и -lrt. Для записи вызовов использована strace. Ввод и результаты соответствуют приведённому выше тесту из четырёх строк.

Команда запуска

```
$ strace -f -qq -s 100 -o trace.log \
  -e
trace=process,read,write,close,openat,newfstatat,fstat,ftruncate,mmap,munmap,unlink,unlinkat,fut
ex,ppoll,clock_nanosleep,rt_sigaction ./parent < input.txt
```

Параметр -f включает дочерние процессы, -o сохраняет журнал, -s ограничивает длину строк в выводе, -e выбирает системные вызовы, -qq убирает служебные сообщения. Первое число строки журнала — PID. Ниже отобраны записи, относящиеся к IPC, запуску потомков, записи результата и очистке. Загрузочные отображения библиотек и повторные проверки wait4 не приводятся. Строки сохраняют PID и результаты фактического запуска. Записи unfinished/resumed объединены без изменения аргументов; фрагменты сгруппированы по назначению и не отражают общий порядок между группами.

Создание общей памяти родителем

Родитель имеет PID 2708. Для обоих каналов журнал показывает открытие именованного объекта, установку размера 4112 байт, MAP_SHARED-отображение и закрытие дескриптора. Размер

структуры включает служебные поля и выравнивание, поэтому больше массива data размером 4096 байт.

```
2708 openat(AT_FDCWD, "/dev/shm/l3_a94_380624c2_0m", O_RDWR|O_CREAT|O_EXCL|O_NOFOLLOW|O_CLOEXEC,
0600) = 5
2708 ftruncate(5, 4112)                 = 0
2708 mmap(NULL, 4112, PROT_READ|PROT_WRITE, MAP_SHARED, 5, 0) = 0xffffa3e80000
2708 close(5)                           = 0
2708 openat(AT_FDCWD, "/dev/shm/l3_a94_382bdbde_1m", O_RDWR|O_CREAT|O_EXCL|O_NOFOLLOW|O_CLOEXEC,
0600) = 5
2708 ftruncate(5, 4112)                 = 0
2708 mmap(NULL, 4112, PROT_READ|PROT_WRITE, MAP_SHARED, 5, 0) = 0xffffa3e7c000
2708 close(5)                           = 0
```

Создание дочерних процессов

В данном запуске потомки получили PID 2709 и 2710. Номера относятся только к этому запуску. Успешные clone соответствуют вызовам fork в исходном коде.

```
2708 clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD,
child_tidptr=0xffffa3e82fd0) = 2709
2708 clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD,
child_tidptr=0xffffa3e82fd0) = 2710
```

Дочерний процесс 1

Процесс 2709 после execve открывает свой shared memory объект и два именованных семафора. write с дескриптором 1 записывает уже отфильтрованный текст в соответствующий выходной файл. Публикация данных родителем через обычные обращения к отображённой памяти отдельных системных вызовов write не создаёт.

```
2709 execve("./child", ["child", "/l3_a94_380624c2_0m", "/l3_a94_380624c2_0e",
"/l3_a94_380624c2_0r"], 0xffffcc2f9788 /* 5 vars */)             = 0
2709 openat(AT_FDCWD, "/dev/shm/l3_a94_380624c2_0m", O_RDWR|O_NOFOLLOW|O_CLOEXEC)              =
3
2709 mmap(NULL, 4112, PROT_READ|PROT_WRITE, MAP_SHARED, 3, 0)               = 0xffffb1976000
2709 openat(AT_FDCWD, "/dev/shm/sem.l3_a94_380624c2_0e", O_RDWR|O_NOFOLLOW|O_CLOEXEC)
= 3
2709 mmap(NULL, 32, PROT_READ|PROT_WRITE, MAP_SHARED, 3, 0)               = 0xffffb1975000
2709 openat(AT_FDCWD, "/dev/shm/sem.l3_a94_380624c2_0r", O_RDWR|O_NOFOLLOW|O_CLOEXEC)
= 3
2709 mmap(NULL, 32, PROT_READ|PROT_WRITE, MAP_SHARED, 3, 0)               = 0xffffb1974000
2709 write(1, "prtng Systms\n", 13)              = 13
2709 write(1, "Prcss cmmnctn\n", 14)              = 14
2709 exit_group(0)         = ?
```

Дочерний процесс 2

Процесс 2710 после execve открывает свой shared memory объект и два именованных семафора. write с дескриптором 1 записывает уже отфильтрованный текст в соответствующий выходной файл. Публикация данных родителем через обычные обращения к отображённой памяти отдельных системных вызовов write не создаёт.

```
2710 execve("./child", ["child", "/l3_a94_382bdbde_1m", "/l3_a94_382bdbde_1e",
"/l3_a94_382bdbde_1r"], 0xffffcc2f9788 /* 5 vars */)             = 0
2710 openat(AT_FDCWD, "/dev/shm/l3_a94_382bdbde_1m", O_RDWR|O_NOFOLLOW|O_CLOEXEC)              =
3
2710 mmap(NULL, 4112, PROT_READ|PROT_WRITE, MAP_SHARED, 3, 0)               = 0xffff86165000
2710 openat(AT_FDCWD, "/dev/shm/sem.l3_a94_382bdbde_1e", O_RDWR|O_NOFOLLOW|O_CLOEXEC)
= 3
2710 mmap(NULL, 32, PROT_READ|PROT_WRITE, MAP_SHARED, 3, 0) = 0xffff86164000
2710 openat(AT_FDCWD, "/dev/shm/sem.l3_a94_382bdbde_1r", O_RDWR|O_NOFOLLOW|O_CLOEXEC)
= 3
2710 mmap(NULL, 32, PROT_READ|PROT_WRITE, MAP_SHARED, 3, 0)               = 0xffff86163000
2710 write(1, "Q d\n", 4)               = 4
2710 write(1, "Frk nd pp\n", 10)              = 10
2710 exit_group(0)                      = ?
```

Ожидание и пробуждение через семафоры

Ниже приведены первые наблюдавшиеся futex-операции. Они обслуживают ожидание и пробуждение внутри POSIX-семафоров. Операции без ожидания могут пройти в пользовательской библиотеке и не появиться отдельной строкой трассировки.

```
2710 futex(0xffff86163000, FUTEX_WAIT_BITSET|FUTEX_CLOCK_REALTIME, 0, NULL,
FUTEX_BITSET_MATCH_ANY)              = 0
2709 futex(0xffffb1974000, FUTEX_WAIT_BITSET|FUTEX_CLOCK_REALTIME, 0, NULL,
FUTEX_BITSET_MATCH_ANY)              = 0
2708 futex(0xffffa3e7e000, FUTEX_WAKE, 1) = 1
2708 futex(0xffffa3e7a000, FUTEX_WAKE, 1) = 1
2709 futex(0xffffb1974000, FUTEX_WAIT_BITSET|FUTEX_CLOCK_REALTIME, 0, NULL,
FUTEX_BITSET_MATCH_ANY)              = 0
2710 futex(0xffff86163000, FUTEX_WAIT_BITSET|FUTEX_CLOCK_REALTIME, 0, NULL,
FUTEX_BITSET_MATCH_ANY)              = 0
```

Завершение и удаление именованных объектов

Родитель получает успешные статусы обоих потомков, удаляет два имени shared memory и четыре имени семафоров, затем завершается с кодом 0. Перед удалением имён локальные отображения снимаются и открытые семафоры закрываются функцией channel_close.

```
2708 wait4(2709, [{WIFEXITED(s) && WEXITSTATUS(s) == 0}], WNOHANG, NULL) = 2709
2708 wait4(2710, [{WIFEXITED(s) && WEXITSTATUS(s) == 0}], WNOHANG, NULL) = 2710
2708 unlinkat(AT_FDCWD, "/dev/shm/l3_a94_380624c2_0m", 0) = 0
2708 unlinkat(AT_FDCWD, "/dev/shm/sem.l3_a94_380624c2_0e", 0) = 0
2708 unlinkat(AT_FDCWD, "/dev/shm/sem.l3_a94_380624c2_0r", 0) = 0
2708 unlinkat(AT_FDCWD, "/dev/shm/l3_a94_382bdbde_1m", 0) = 0
2708 unlinkat(AT_FDCWD, "/dev/shm/sem.l3_a94_382bdbde_1e", 0) = 0
2708 unlinkat(AT_FDCWD, "/dev/shm/sem.l3_a94_382bdbde_1r", 0) = 0
2708 exit_group(0)                    = ?
```

Связь исходного кода и трассировки

В этой Linux-среде POSIX shm_open реализован через открытие объекта в /dev/shm, а sem_open создаёт отображаемые объекты с префиксом sem. Это внутреннее представление POSIX IPC в Linux; исходный код не заменяет shm_open обычным открытием файла в /tmp. Именно дескриптор shared memory отображается через MAP_SHARED. Пользовательские записи в data после mmap являются обращениями к памяти и не появляются в strace как отдельные write.

Семафорные операции могут завершаться без входа в ядро, если разрешение уже доступно. При блокировке библиотека использует futex, поэтому отдельного системного вызова sem_wait в журнале нет. На aarch64 fork наблюдается как clone, execl — как execve, waitpid — как wait4, а poll — как ppoll. Имена обёрток и системных вызовов зависят от платформы.

close закрывает дескриптор процесса, но не уничтожает готовое отображение. munmap освобождает локальное отображение; unlinkat удаляет имя IPC-объекта в данном Linux-журнале. После обработки EOF оба child завершаются с кодом 0, parent получает их статусы и удаляет два shared memory объекта и четыре семафора. Проверка выходных файлов подтверждает результат, а пустой /dev/shm — очистку именованных ресурсов в данном контейнере.

Вывод

Реализован обмен строками между родителем и двумя потомками через именованную POSIX shared memory и mmap с MAP_SHARED. Два семафора на каждый слот обеспечивают передачу данных без перезаписи ещё не обработанного сообщения. Программа сохраняет вариант 18 первой лабораторной: распределяет строки по чётности и удаляет латинские гласные в потомках. Длинные строки передаются фрагментами, конец ввода — отдельным сообщением. Реальные запуски, 23 интеграционные проверки на каждой из двух платформ и Linux-трассировка подтвердили корректную обработку текста и освобождение ресурсов в проверенных сценариях.